REVISION HISTORY

Revision No./Version No.	Date	Prepared by	Approved by	Details of Change
1.0	20-June-2024	Lakshiman Chodankar (Software Developer)	Anant Yende (CTO)	

OBJECTIVE

Ensure efficient collaboration, version control, and code quality within development teams. Ensure daily tracking and maintaining common repository of the source code, developed/maintained within the organization. Ensure developers codes are backed up on a daily basis to avoid loss due to hardware failure, etc.

PARTICIPANTS

1. **Developers:** Responsible for creating developer branches, writing code, submitting merge requests and daily commits to respective developer branch (Developer).

2. **Team Leads (TL):** Responsible for creating Project Repository, assigning privileges to users, oversee code quality, conduct reviews, and manage (review/approve/reject) merge requests (Maintainer Role).

3. Dev-Ops Team: Manage CI/CD pipelines and ensure successful deployment of applications (Maintainer Role).

PROCEDURE

Refer the Git Branching Tree Structure as per the Fig 1.0 of the Annexure along with the definitions.

1. <u>Procedure to setup code repository by TL</u>

- Team Leads should create a new repository in GitLab and give a name based on Project Name, discussing with Solution Architect, with following format. <project_shortname3char>-<projectname>-<technology>.

- Define repository with private visibility.
- Add members/developers to the repository with TL having the "Maintainer" role and the Developers having "Developer" role. Also add Dev-ops engineer with "Maintainer" role.
- Set an expiry date of one year from the date each member is added.
- Add a README file to document the project's information and setup instructions.
- Create basic branches (master, staging, development) for a repository, if not created, referring to the Git Branching Tree Structure mentioned as per the Fig 1.0 of the Annexure.

2. Procedure to Access/Commit to code repository by a Developer

- Developer to create a branch with her/his name towards the feature or development branch, if not already created.
- Clone the repository with the created branch, on local system.
- Developer to work on the pulled code.
- Carry out daily commits in the self branch mentioning clear and descriptive commit messages summarizing changes.
- Everyday e.o.d, changes done in the developer or feature branch have to be pushed to the remote repository using git push.
- Once development is complete, the developers have to clean their code, then commit and push to the developer branch or feature branch specific to developer, before merging with the development/feature branch.
- The developers have to review their changes before committing, so as to ensure that only the required stable code is being committed, and code/files which are not related to a feature/sub-feature are not committed.
- The developer has to make sure to remove any unnecessary extra lines or spaces that are added in a code, and also remove any unwanted files/code which might have been temporarily changed.
- The developer has to send merge request (MR) to TL from developer or feature branch as per project configuration, and may also additionally add a reviewer for code review.

3. Procedure to Merge Code & Deploy by TL/Maintainer

- The TL has to review the changes in the MR and address comments and suggestions from reviewers before merging.
- TL has to merge the developer or feature branch into the development branch after complete review is done and if satisfactory.

- TL has to check for any conflicts before merging and resolve them before merging.
- TL should merge the feature branches on the pre-production branch as per the sprint plan, and the same to be given for testing after the planned modules are completed. After clearance, the TL to merge the pre-production branch in the master. If the pre-production branch is not created, TL should merge the feature branches on the master branch as per the sprint plan.
- TL to decide and delete the feature branches post-merge on master branch, to keep the repository clean. The TL to decide if a certain feature branch is not to be deleted.
- Monitor deployment logs for errors and ensure successful application launch.
- In-case of failure in deployment, TL to revert back to the previous stable master branch, if necessary consulting Project Manager and CTO.

ANNEXURE

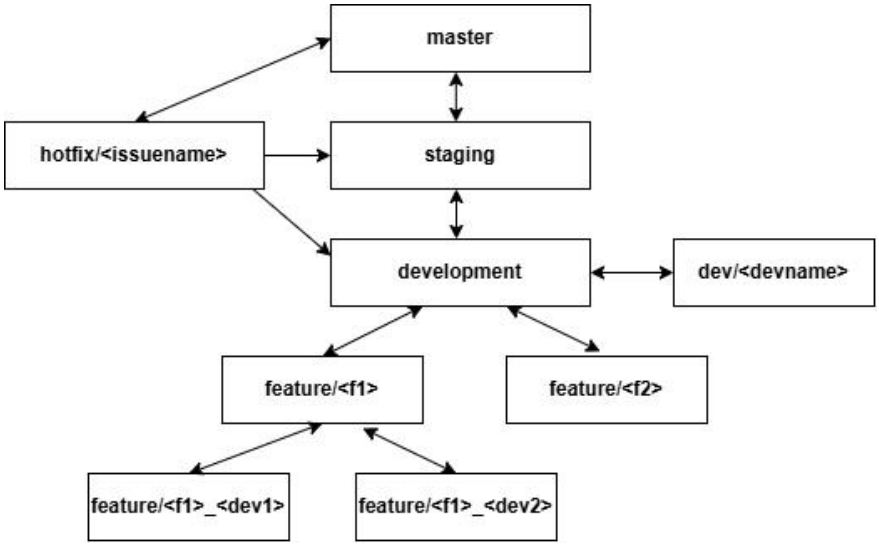

Fig 1.0: Git branching tree structure

Definitions and Branching Tree Structure

- Developer and TL has to follow the Branching strategy as per the Fig 1.0 of the Annexure.
- **Gitlab:** A web-based Git repository manager providing features for version control, CI/CD, and project management.
- **Repository (Repo):** A storage location for code, including all versions and changes.
- **Branch:** A parallel version of the codebase created for development, allowing for isolated changes.

- **Merge Request (MR):** A request to merge changes from one branch into another, typically requiring code review.
- **CI/CD:** Continuous Integration/Continuous Deployment, a practice to automate code integration and deployment.
- **Main Branch:** Team Lead has to create this branch which will have stable version of the production code. It has to be named as "main" or "master" branch. Merge to this branch should be allowed to be done only by a Maintainer.
- **Development Branch:** Team Lead has to create a branch for ongoing development, and name it "development". This branch will have all code which is in active development.
- **Developer Branch:** Developer has to create this branch for active development pertaining to a developer. This branch has to be created from the master branch following the naming convention, dev/<developer-name>. The developer has to regularly merge the master branch in the developer branch so that it remains synced with the latest changes. Also a developer has to regularly commit his stable changes everyday in this branch and push changes on git server at e.o.d.
- **Feature Branch (Optional):** A branch for each feature or bug fix, following the naming convention: "feature/<featurename>" or "bugfix/<bugdescription>". This branch has to be created by the Developers in consultation with the Team Lead. If two or more developers are working on the same feature or task simultaneously, first create a new branch from the development or the appropriate parent branch (e.g. feature/<featurename>). Then use specific branch names that identify who is working on what, for example: "feature/<featurename>-<developer1name>", "feature/<featurename>-<developer2name>".
- **Testing Branch (Optional):** A branch to be created by TL for every testing which is done by the Testing Team. A TL can optionally create phase-wise branches for the testing being put up. Use the following format for creating the branches, "testing/<jiraticketid>-<phasenumber>".
- **Pre-production Branch (Optional):** A branch to be created by TL from the master branch, which would have all the feature branches merged for a particular sprint plan. This branch would be given for Testing before going Live. The naming convention for a pre-production branch would be "staging".
- **Hotfix Branch (Optional):** Create hotfix branches from the master branch to fix any production issue, following the naming convention, hotfix/<bugfixname>. This branch has to be created by the developer. TL to ensure that the hotfix branches should be merged back into both master and development branch.